

PROGRAMAÇÃO ORIENTADA A OBJETOS

LISTA #02 DE EXERCÍCIOS

Pontos Importantes...

- Em algumas empresas, o UML é amplamente utilizado. Às vezes, o programador o recebe já pronto e completo, devendo somente preencher a implementação e seguir, à risca, o UML. O que você acha dessa prática? Quais as vantagens e desvantagens?
- Se uma classe só tem atributos e métodos estáticos, que conclusões podemos tirar? O que lhe parece um método estático em casos como esses?
- No caso de atributos booleanos, pode-se usar, no lugar do `get`, o sufixo `is`. Desse modo, caso tivéssemos um atributo booleano `ligado` em vez de `getLigado`, poderíamos ter `isLigado`.

Exercícios

1. O que é necessário fazer para **garantirmos** que os atributos da classe `Conta` não sejam acessados de forma direta em outra classe a qual não seja a própria classe `Conta`?
2. Após deixar os atributos da classe `Conta` com acesso restrito (privado), tente criar uma `Conta` na classe `TestaConta` dentro do `main` e modificar ou ler os atributos da conta criada. O que acontece?
3. Crie apenas os *getters* e *setters* necessários na sua classe `Conta`. Pense sempre se é preciso criar cada um deles.

Não copie e cole! Aproveite para praticar a sintaxe. Logo, passaremos a usar a própria IDE para gerar esses métodos.

4. Repare que o método `calculaRendimento` parece também um *getter*. Aliás, seria comum alguém nomeá-lo de `getRendimento`. *Getters* não precisam apenas retornar atributos, eles podem trabalhar com esses dados.
5. Altere suas classes que acessam e modificam atributos de uma `Conta` para utilizar os *getters* e *setters* recém-criados.





6. Faça com que sua classe `Conta` possa receber, opcionalmente, o nome do titular da `Conta` durante a criação do objeto.
7. (Opcional) Adicione um atributo, na classe `Conta` de tipo `int`, que se chama `identificador`. Este deve ter um valor único para cada instância do tipo `Conta`. A primeira `Conta` instanciada tem `identificador` 1, a segunda, 2, e assim por diante. Você deve utilizar os recursos aprendidos aqui na resolução desse problema.
8. Crie um *getter* para o `identificador`. Devemos ter um *setter*?
9. (Opcional) Como garantir que datas como 31/2/2021 não sejam aceitas pela sua classe `Data`?
10. (Opcional) Imagine que tenhamos a classe `PessoaFisica` a qual tem um CPF como atributo. Como garantir que alguma pessoa física tenha CPF inválido e tampouco seja criada uma `PessoaFisica` sem CPF inicial? (Suponha que já exista um algoritmo de validação de CPF: este deve passar por um método `valida(String x)...`)

Desafios

1. Por que esse código não compila?

```
class Teste {  
    int x = 37;  
    public static void main(String [] args) {  
        System.out.println(x);  
    }  
}
```

2. Imagine que haja uma classe `FabricaDeCarro`, e quero garantir que só exista um objeto desse tipo em toda a memória. Não há uma palavra-chave especial para isso em Java. Então, teríamos de fazer nossa classe de tal maneira que ela respeitasse essa nossa necessidade. Como faríamos isso? (Pesquise: *singleton design pattern*.)

